决策层和执行层

视觉 = 感知层

输出位置

```
物体类别: "杯子"
位置: x=0.5m, y=0.2m, z=0.0m
姿态: roll=0°, pitch=0°, yaw=90°
```

1. 物体类别
2. x,y,z坐标
3. 姿态 `(roll, pitch, yaw)`

给到高层决策

1️⃣ 高层 = 决策层（Decision Layer）

功能：决定机器人“做什么任务”。

- 对应你的抓取任务：**选择抓哪个物体、抓取顺序、是否需要抓取动作**
- 输入：环境状态、目标任务信息
- 输出：任务目标，例如：

```
抓取物体A
或
移动到充电区
```

2️⃣ 中层 = 规划层（Planning Layer）

功能：决定“怎么做”，也就是实现高层决策所需的路径或动作。

- 对应抓取任务：
  - **路径规划**：机械臂如何到达抓取点
  - **局部避障**：避免碰撞桌面障碍物或机械臂自身
  - **逆向运动学（IK）**：末端执行器姿态 → 各关节角度
- 输入：高层决策输出（目标物体位置）、环境地图或传感器信息
- 输出：关节轨迹、末端执行器运动路径、速度指令

  末端执行器的抓取姿态：位置 `(x, y, z)` + 姿态 `(roll, pitch, yaw)`

  关节轨迹（可选）：每个关节角度随时间变化

  速度/加速度限制

  夹爪的开关和闭合

✅ 核心：**计划机械臂运动的具体路线和动作**。

3️⃣ 低层 = 控制层（Control Layer）

功能：决定“电机如何动作”，把规划转化为实际运动。

- 对应抓取任务：
  - 各关节按规划轨迹运动（PID、LQR、MPC）
  - 控制夹爪动作（开/闭）
  - 安全控制（碰撞检测、限速）
- 输入：规划层输出（关节角度/速度轨迹）
- 输出：具体电机指令
- 核心：**把规划执行到物理世界**。

代码块

```
[高层决策] → 决定抓哪个物体
          ↓ 输出：抓取目标
[中层规划] → 计算末端姿态 + 路径
          ↓ 输出：关节轨迹
[低层控制] → PID/LQR/MPC 控制关节
          ↓ 输出：电机指令
[机械臂执行] → 移动到物体，夹爪闭合，抓取
```

```
8                ↓ 反馈：抓取成功/失败
```

名词解释

高层决策

- **规则（Rule）**：最简单的方法，用 if…else 判断要做什么
  - 例：如果检测到杯子 → 去抓
- **FSM（Finite State Machine，有限状态机）**：用状态和状态转移控制行为
  - 例：状态有“空闲→抓取→放置”，根据条件切换
- **行为树（Behavior Tree）**：把任务分成树状节点，每个节点是动作或条件
  - 直观、易扩展
- **任务规划（Task Planning）**：计划完成多个任务的顺序
  - 例：先抓A再抓B
- **大模型（LLM/AI模型）**：用AI决策“做什么”
  - 工程落地成本高，不一定用

中层规划

- **A***：常用路径规划算法，找到最短路径
- **RRT（Rapidly-exploring Random Tree）**：快速生成可行路径，适合机械臂
- **DWA（Dynamic Window Approach）**：局部避障算法，实时避开障碍
- **TEB（Timed Elastic Band）**：局部轨迹优化，避障又平滑
- **MPC（Model Predictive Control）**：根据模型预测未来动作，规划最优轨迹

低层控制

- **PID（比例-积分-微分控制）**：最常见的控制器，让关节按规划轨迹运动

- **LQR（线性二次调节器）**：高级控制器，用数学模型优化动作
- **MPC**：也可以用于低层，实时控制关节
- **强化学习控制（RL）**：让机器人通过学习控制关节，落地复杂

ik：逆向运动学

1️⃣ IK 是什么?

IK = Inverse Kinematics（逆向运动学）

- **作用**：把末端执行器的位置和姿态 `(x, y, z, roll, pitch, yaw)` 转换成机械臂每个关节的角度 `[θ1, θ2, … θn]`。
- **为什么需要**：
  - 高层决策或视觉识别只告诉你 **抓取点的位置** 和姿态
  - 机械臂控制需要的是 **每个关节角度** 才能执行动作
- **对应的概念**：
  - **Forward Kinematics (FK，正向运动学)**：已知关节角度 → 计算末端执行器位置
  - **Inverse Kinematics (IK，逆向运动学)**：已知末端位置 → 计算关节角度

示意图理解：

```
目标末端位置 + 姿态  -->  IK  -->  各关节角度  -->  机械臂动作
```

落地组合

- **高层决策**：行为树 / FSM
- **中层规划**：路径规划 A* / RRT + 局部避障 DWA / TEB
- **低层控制**：PID 或 MPC

lerobot对你的落地方案影响

1. **低层控制**：直接用 LeRobot API 控制 6 个关节和夹爪 。
   a. 关节控制
   b. 夹爪控制
   c. 安全检测
2. **中层规划**：
   - 可以在 LeRobot 提供的接口上实现 RRT / A* / DWA 轨迹规划
   - IK 可能用 LeRobot 自带函数，或者自己实现
3. **高层决策**：行为树/FSM 完全和 LeRobot 解耦

- 高层只关心“抓哪个物体”，不需要管硬件接口

选择路线一

两条路线的**命令下达流程 + 驱动链路:**

路线 1：标准 ROS2 + MoveIt2 + ros2_control

代码块

```
高层（用户/行为树/FSM）
↓ 目标位姿
MoveIt2
↓ 规划关节轨迹 + IK
ros2_control HardwareInterface
↓ 关节命令
joint_trajectory_controller
↓ 控制信号
LeRobot ROS Node（驱动 SO Arm 101）
↓ 电机动作
机械臂执行
↓ 状态反馈
joint_state_broadcaster
↓ 反馈给 MoveIt2 / 高层
```

代码块

```
1️⃣ 高层决策（用户/行为树/FSM）
   输入：视觉识别输出（物体类别 + 位置 + 姿态）
   输出：抓取目标末端位姿 target_pose = [x, y, z, roll, pitch, yaw]
        ↓
2️⃣ 中层规划（MoveIt2）
   输入：target_pose
   功能：
      - IK（Inverse Kinematics）求解关节角
      - 路径规划（RRT / OMPL）
      - 避障检测
   输出：joint_trajectory（时间序列关节角）
        ↓
3️⃣ ROS 标准控制器层
   ├─ ros2_control HardwareInterface
   │    作用：把 MoveIt2 输出的轨迹转换成硬件可执行命令
   │    输入：joint_trajectory
```

```
    │    输出：控制器可识别的关节命令
    ↓
    ├─ joint_trajectory_controller
    │    作用：按轨迹周期控制关节角度/速度
    │    输出：实际关节电机指令
    ↓
4️⃣ LeRobot ROS Node
   作用：驱动 SO Arm 101，执行关节动作 + 夹爪动作
        ↓
5️⃣ 机械臂执行
   输出：物理运动 + 抓取动作
        ↓
6️⃣ 状态反馈
   ├─ joint_state_broadcaster
   │    作用：周期性发布关节状态（角度/速度）
   │    输出：关节状态 Topic
   ↓
7️⃣ 高层/MoveIt2接收状态反馈
   功能：
      - 高层行为树判断抓取是否成功
      - MoveIt2 可用于下一步规划
```

流程说明

1. **高层决策**
   - 用户或行为树决定抓取目标 `(x, y, z, roll, pitch, yaw)`
2. **MoveIt2**
   - 使用 IK 将末端位姿转换为关节角 `[01…06]`
   - 生成平滑轨迹
3. **ros2_control HardwareInterface**
   - 将 MoveIt 轨迹转换为控制器理解的命令
4. **joint_trajectory_controller**
   - 根据轨迹周期发送关节角度/速度指令
5. **LeRobot ROS Node**
   - 接收关节命令 → 转发给机械臂电机
   - 控制夹爪开/闭
6. **机械臂执行**

- 机械臂关节按照命令运动

7. **joint_state_broadcaster**
   - 周期性发布关节状态（角度、速度）
   - MoveIt2 和高层可以实时获得执行状态

1️⃣ 节点划分（每个功能对应一个 ROS 节点）

层级	节点名称	功能	ROS接口
感知层	`vision_node`	AI视觉识别，输出物体类别 + 位姿	发布 Topic `/detected_object`
高层决策	`behavior_tree_node`	根据视觉结果选择抓取目标和策略	订阅 `/detected_object`，发布目标位姿 `/target_pose`
中层规划	`moveit_planning_node`	MoveIt2 生成关节轨迹、IK、避障	订阅 `/target_pose`，发送轨迹到 Action `/arm_controller/follow_joint_trajectory`
低层控制	`arm_action_server_node`	接收轨迹 → LeRobot执行机械臂动作	Action Server `/arm_controller/follow_joint_trajectory`，发布 `/joint_states`
状态反馈	joint_state_broadcaster	周期发布关节角度状态	Topic `/joint_states` → 高层/MoveIt2闭环反馈

路线 2：简化桥接方案（MoveIt → action server → LeRobot）

代码块

```
高层（用户/行为树/FSM）
↓ 目标位姿
MoveIt2
↓ 规划关节轨迹 + IK
你的 ROS Action Server 节点
↓ 转发轨迹
LeRobot API
↓ 电机动作
```

```
机械臂执行
↓ 状态反馈
Action Server 节点
↓ 反馈给 MoveIt2 / 高层
```

流程说明

1. **高层决策**
   - 同样输出抓取目标
2. **MoveIt2**
   - 生成关节轨迹 `[θ1…θ6]`
3. **Action Server 节点**
   - 订阅 MoveIt 发布的轨迹
   - 可以加安全检查（关节限位、碰撞约束）
   - 调用 LeRobot API 控制机械臂
4. **LeRobot API**
   - 直接发送电机指令 + 夹爪动作
5. **机械臂执行**
   - 按轨迹运动
6. **状态反馈**
   - Action Server 周期性读取机械臂状态
   - 返回给高层或 MoveIt2

对比总结

项目	路线1（标准ROS2）	路线2（简化桥接）
控制器	joint_trajectory_controller	Action Server 节点自己实现
中层规划输出	MoveIt2 → ros2_control	MoveIt2 → Action Server
驱动机械臂	LeRobot ROS Node 接收 controller 指令	LeRobot API 被 Action Server 调用
状态反馈	joint_state_broadcaster	Action Server 读取机械臂状态
优点	标准化、兼容 MoveIt2 / Gazebo / RViz，调试方便	简单快速落地，少配置 ROS2 控制器
缺点	配置复杂，调试 controller 层麻烦	没有标准 controller 层，安全性和扩展性略低

💡 核心理解：

- **中层规划** = MoveIt2（IK + 轨迹生成）
- **低层控制** = ros2_control + joint_trajectory_controller 或 Action Server + LeRobot API
- **两条路线的区别** = 是否使用 ROS 标准控制器层

代码块

```
MoveIt2（中层规划输出轨迹）
        ↓
ros2_control HardwareInterface（硬件抽象层）
        ↓
joint_trajectory_controller（关节轨迹执行）
        ↓
机械臂电机动作（真实执行）
```

使用 **ROS2 + MoveIt2** 结合标准控制器，相比直接调用 LeRobot API，通常能实现 **更精准、可控和稳定的机械臂运动**。

方面	原理 / 举例
轨迹规划精度	MoveIt2 内置的运动规划算法（RRT、OMPL）+ IK 求解，生成平滑、连续的关节轨迹，避免机械臂抖动或跳跃
闭环控制	ROS2 的 joint_trajectory_controller 支持 PID/LQR/MPC 控制，可自动补偿关节误差，保持轨迹精度
碰撞检测和避障	MoveIt2 内置碰撞检查，规划轨迹时自动避开障碍物，减少因碰撞导致的轨迹偏差
可视化和调试	RViz 显示关节角度、轨迹，便于调整规划参数，减少人为误差

特性	直接 LeRobot API	ROS2 + MoveIt2
轨迹平滑度	需要自己插值	MoveIt2 内置平滑
避障能力	自己实现	MoveIt2 + collision checking 自动避障
PID 控制	可实现，但需手动	joint_trajectory_controller 自动闭环
可视化调试	有限	RViz 可直观显示轨迹和碰撞
系统扩展	单机控制，扩展麻烦	多模块协作、容易添加传感器